The iconv module is a wrapper around the system libiconv library and is used for encoding conversion and encoding-aware equivalents of core functions like strpos() and substr(), but using character width rather than byte width. As such, there are equivalents in the mbstring module (and more besides).

The intl module is far more comprehensive: it wraps the ICU (International Components for Unicode) library and offers a large variety of tools for all your internationalization needs.

iconv

This module offers a small selection of functions, given below:

Table 8-40. *iconv extensions functions*

iconv()	Convert string encoding
iconv_get_encoding()	Get internal config of input, output, internal, or all
iconv_mime_decode()	Decode a single MIME header
iconv_mime_decode_headers()	Decode multiple MIME headers
iconv_mime_encode()	Generate a MIME header with encoding
iconv_set_encoding	Set input, output, and internal encoding
iconv_strlen()	Get character count of string according to encoding
iconv_strpos()	As strpos() but using character width, encoding aware
iconv_strrpos()	As strrpos() but using character width, encoding aware
iconv_substr()	As substr() but using character width, encoding aware
ob_iconv_handler()	Convert encoding, using output buffer handler

There's little here that isn't already covered by the mbstring functions; the official documentation can be found here: https://www.php.net/manual/en/book.iconv.php

intl

The intl extension consists of several APIs that provide functionality for various tasks such as collation, number formatting, and date and time formatting. All of the APIs offer both functional and object-oriented interfaces, and all input must be in UTF-8 (all output will also be UTF-8 encoded). Here are a couple of examples: